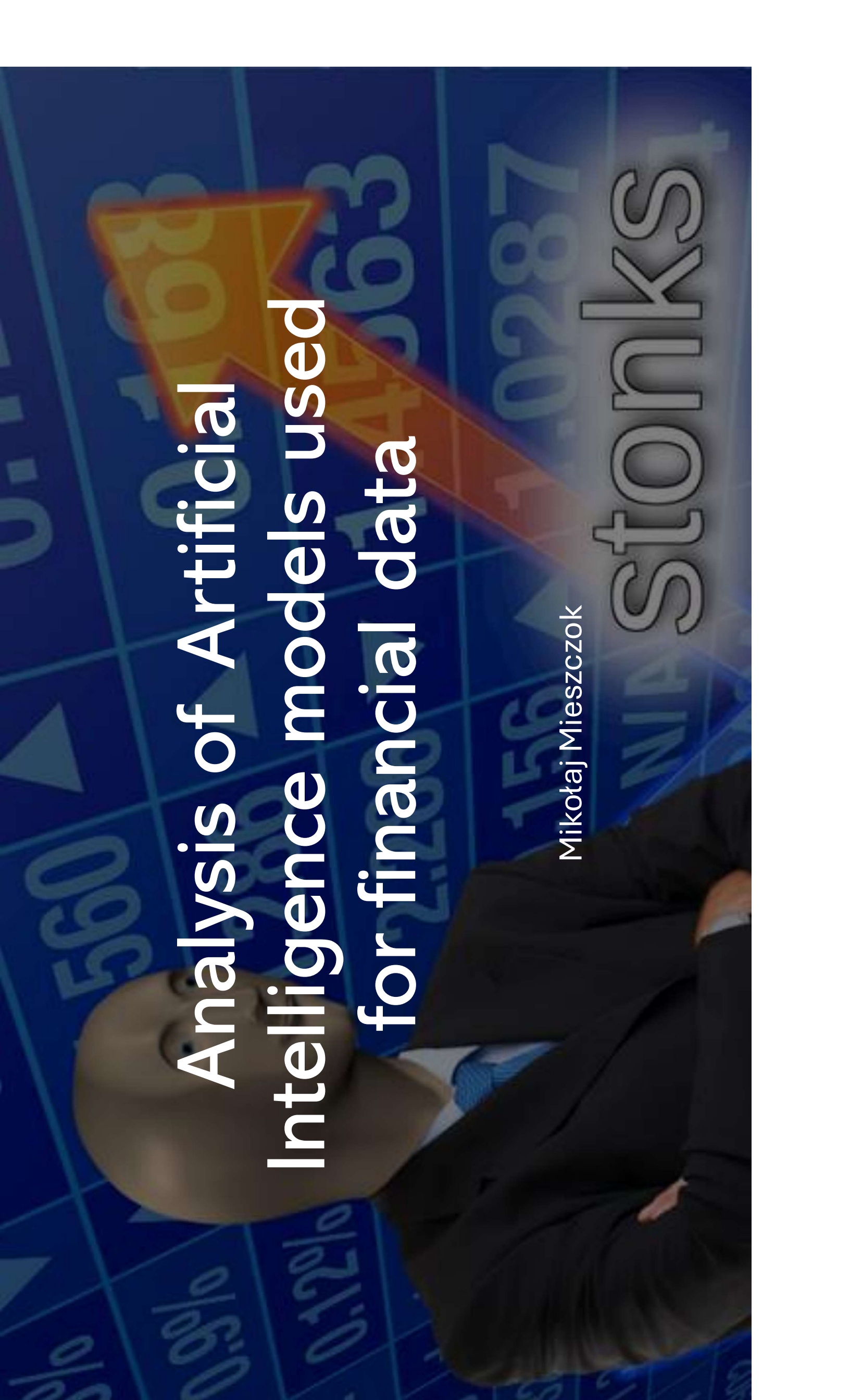


A man in a dark suit, white shirt, and blue tie stands with his arms crossed. The background is a blue grid with various financial symbols, including percentages (0.9%, 0.12%), numbers (560, 2,207, 156, 1,008, 4,263, 1,0287), and arrows. A large, 3D orange arrow points upwards and to the right. The word 'Stonks' is written in a large, outlined font on the right side.

Analysis of Artificial Intelligence models used for financial data

Mikołaj Mieszczok

Stonks

Motivation behind the project

Stock market is a devious playing field. The only ways to generate risk-free profit is either:

- Have a very large capital
- Have access to insider information
- Have connections in the government

For an average person the chance of having access to even one of those methods is very low, not to mention one of them is illegal. Unfortunately, or fortunately for us, the law does not apply to the rich and powerful so we can exploit points 2 and 3. And after a while hopefully point 1 as well.

How the project functions

- <https://www.capitoltrades.com/trades?page=1> is a website that displays trades of US politicians in compliance with the Stock Act of 2012 issued by Obama to combat insider trading. Politicians have to report their trades up to 45 days since making them. It contains 34520 individual trades dating back to June 2022.

Creating a Data-Sheet

Using a web-scraper and a python library called yfinance I created 2 excels: One containing the politicians' names, the stocks they traded, amount of time they took to report the trade as well as how the stock performed in the 60 days prior to publishing of the trade. The second one contained the stocks performance 60 days into the future. Using the second excel I calculated the index of the maximum value of the stock in the future. I saved the index if the value was higher by 5% than when the report was published, or 0'ed it when it was not.



Code for creating the database

```
for x in range(1, 3):
    url = url_start + str(x)
    page = urlopen(url)
    html_bytes = page.read()
    html = html_bytes.decode("utf-8")
    soup = BeautifulSoup(html, 'html.parser')
    politician_links = soup.find_all('a', class_='text-txt-interactive')
    stock_links = soup.find_all('span', class_='q-field issuer-ticker')
    filed_after_links = soup.find_all('span', class_='reporting-gap-tier--1', 'reporting-gap-tier--2', 'reporting-gap-tier--3', 'reporting-gap-tier--4'])
    whether_buy_links = soup.find_all('span', class_='q-field tx-type--buy', 'q-field tx-type--sell', 'q-field tx-type tx-type--buy has-asterisk',
    date_filed_links = soup.find_all('div', class_='text-size-3 font-medium')
    amount_bought_links = soup.find_all('span', class_='mt-1 text-size-2 text-txt-dimmer hover:text-foreground')
    year_bought_links = soup.find_all('div', class_='text-size-2 text-txt-dimmer')

for row in range(1, 10):
    try:
        print(row)
        row = str(row)
        not_filtered = yf.Ticker(str(ws["B" + row].value).split(".")[0]).history(start = str((ws["D" + row].value
        #print(not_filtered)
        if not_filtered.empty:
            continue
        filtered = not_filtered["Open"]
        for col, price in enumerate(filtered):
            worksheet.write(columns[col] + row, price)
        except Exception as e:
            print(f"Error at row {row}: {e}")
    workbook.close()
```


POLITICIAN	TRADED	ISSUER	PUBLISHED	TRADED	FILED AFTER	OWNER	TYPE	SIZE	PRICE	
 Markwayne Mullin Republican Senate OK	 Adobe Inc ADBE:US	18:15 Yesterday	13 May 2025	29 days	 Joint	BUY*	 15K-50K	\$397.40		
 Markwayne Mullin Republican Senate OK	 Applied Materials Inc AMAT:US	18:15 Yesterday	13 May 2025	29 days	 Joint	BUY*	 15K-50K	\$173.03		
 Markwayne Mullin Republican Senate OK	 CSX Corp CSX:US	18:15 Yesterday	13 May 2025	29 days	 Joint	BUY*	 15K-50K	\$30.54		
 Markwayne Mullin Republican Senate OK	 Chevron Corp CVX:US	18:15 Yesterday	13 May 2025	29 days	 Joint	BUY*	 50K-100K	\$142.35		
 Markwayne Mullin Republican Senate OK	 ConocoPhillips COP:US	18:15 Yesterday	13 May 2025	29 days	 Joint	BUY*	 15K-50K	\$94.17		

Further usage

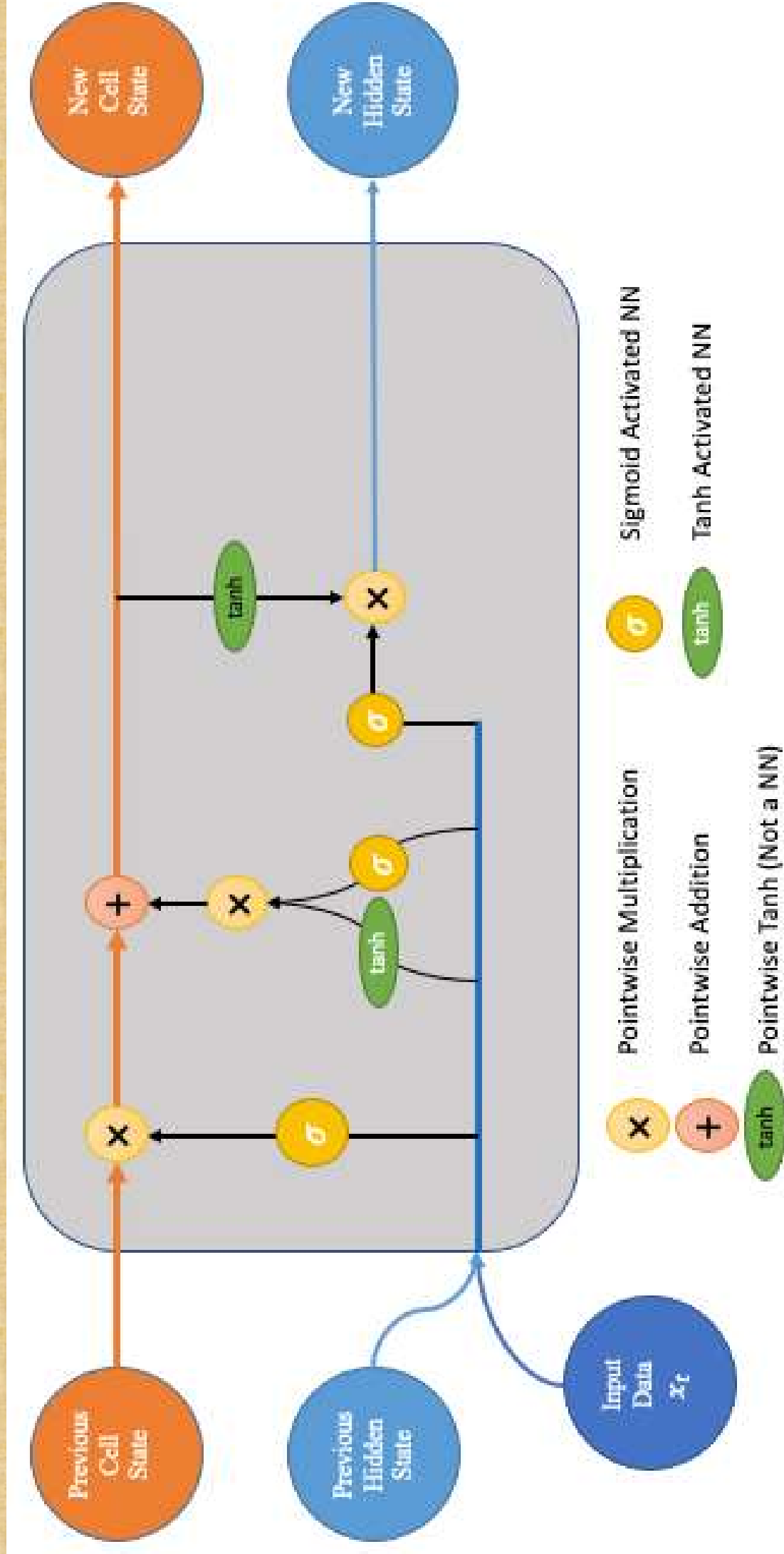
With the training set created all we have to do is provide new data to predict if the stock is worth buying. After the market closes I run the slightly modified webscraper to get all the trades that happened on that very day, then I run the script that gets me the past prices of the stock and then I pass the outcome into the model.

Profitability

I cut out a 500 trade sample from the training database, spanning 42 days and run 5 different models for it:

- Random Forest
- LGBM
- LSTM 200 epoch
- LSTM 400 epoch
- LSTM 600 epoch

Neural Network - LSTM



LSTM CODE

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_test = test_df.drop(columns=["D"], errors='ignore')
X_test_scaled = scaler.transform(X_test)

X_resaped = X_scaled.reshape((X_scaled.shape[0], 1, X_scaled.shape[1]))
X_test_resaped = X_test_scaled.reshape((X_test_scaled.shape[0], 1, X_test_scaled.shape[1]))

model = Sequential()
model.add(LSTM(64, input_shape=(1, X_resaped.shape[2]), return_sequences=False))
model.add(Dropout(0.5))
model.add(Dense(64, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

model.fit(X_resaped, y_cat, epochs=600, batch_size=32, validation_split=0.2)

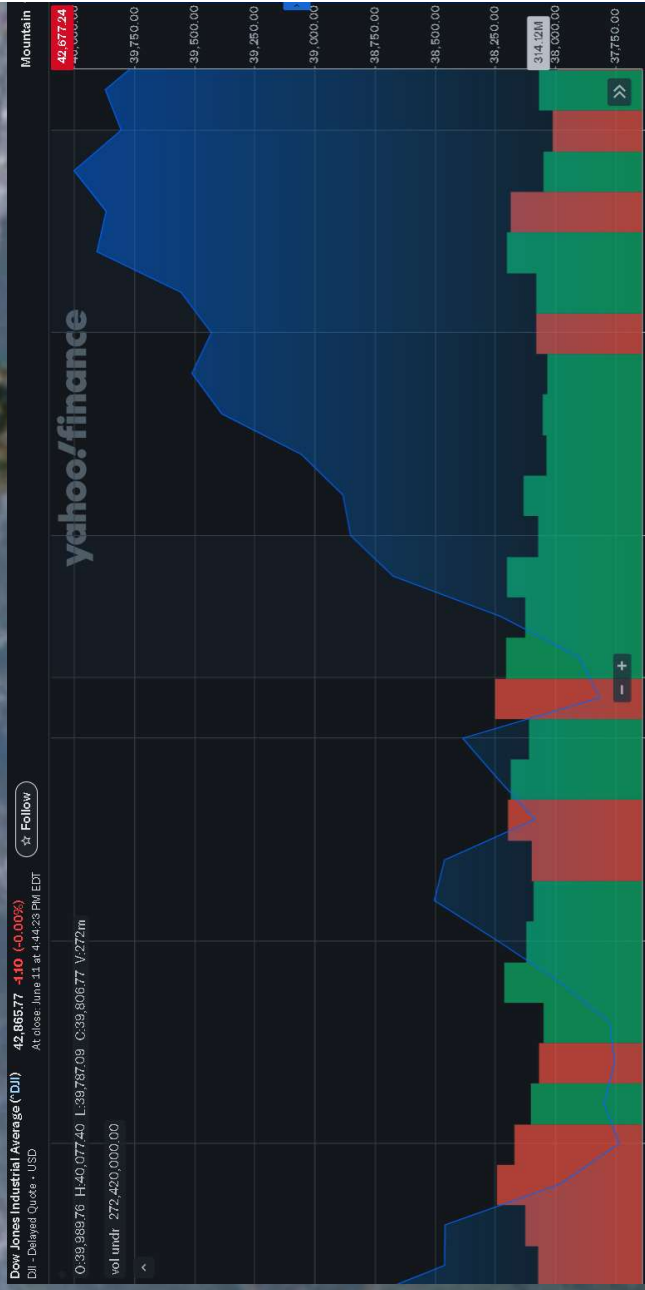
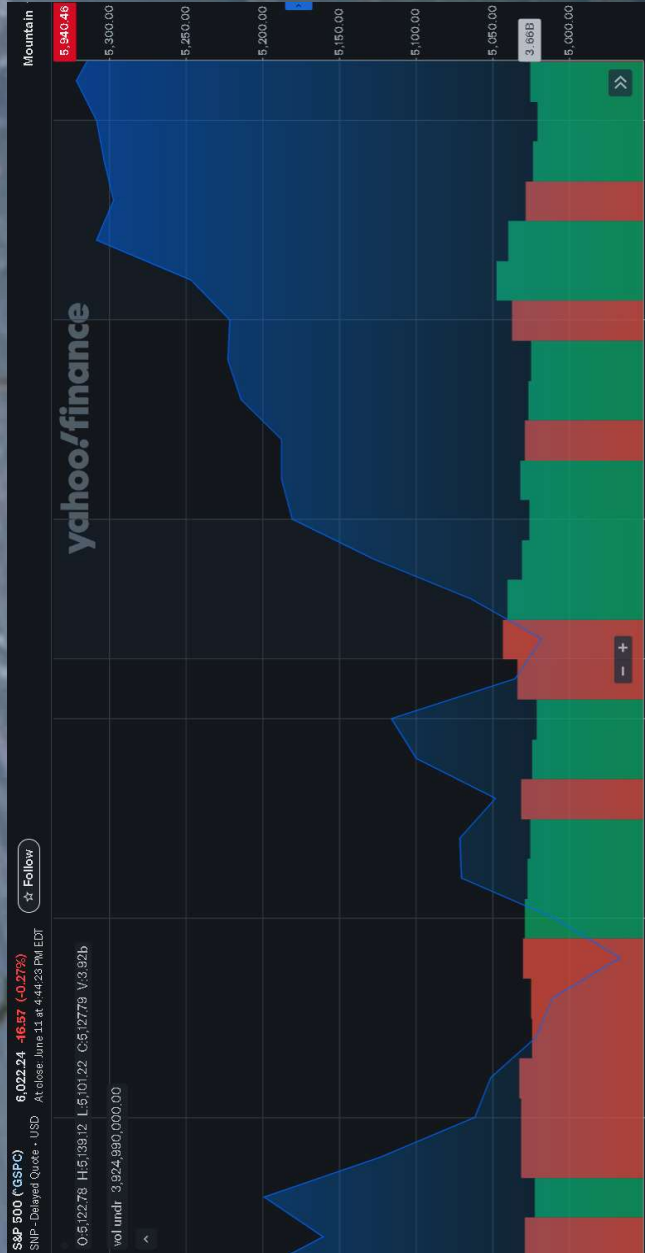
probas = model.predict(X_test_resaped)
confidences = np.max(probas, axis=1)
predicted_classes = np.argmax(probas, axis=1)
```


Profit

	RF	LGBM	LSTM 200	LSTM 400	LSTM 600
Profit	653.039	1491.4	-62.556	4811.9833	313.3388954
nr of trades	25	70	19	28	35
% Profit	2.61216	2.13057	-0.32924	17.185655	0.895253987
avr trade profit	3.0565	2.2667	-0.36798	18.507628	0.949511804

Comparison to other forms of investing

If we were to buy a safe index in the same time period as our sample it would generate us a profit of 7,83% for S&P500, 3,86% for Dow Jones



Future improvements



FUTURE AHEAD

- Test other forms of Artificial Intelligence
- Add one more column to the Training Database to signalise whether the politician bought or sold the stock and add the ability to train on sold stocks as well
- Automatize the whole process

THANK YOU FOR YOUR ATTENTION

Confused stonks